ServiceM8 Integration Guide

See also: [TESTING_GUIDE.md]

1. What This Integration Does

The Newo AI Agent integrates with ServiceM8 so your AI can act as a field service receptionist for your business.

Once connected, the AI agent can:

- check staff availability for requested service dates
- create field service jobs (plumbing, electrical, HVAC, etc.)
- cancel jobs
- reschedule jobs to a new date/time
- look up existing job status and schedule details
- create price quotes / estimates
- auto-add default FSM scenarios to Canvas
- refresh static data (staff, categories, materials) in the background

In plain terms: the AI can talk to customers about availability and bookings while ServiceM8 acts as the job management backend.

Manager Quick View

Question	Short answer
What business problem does this solve?	Lets the AI handle job scheduling, cancellation, rescheduling, and status lookup without a human dispatcher.
Who is this for?	Field service businesses already using ServiceM8 (plumbers, electricians, HVAC, cleaners, handymen).
What is needed before setup?	ServiceM8 API Key plus default staff UUID.
How long does setup usually take?	5-10 minutes if API key is ready.
How do we know it worked?	Publish completes, `servicem8_staff` attribute is populated with JSON, agent returns available slots in a test conversation.
What should we test first?	Availability check, then booking.

Best Fit / Not a Fit

Best fit:

- field service job scheduling (plumbing, electrical, HVAC, cleaning)
- AI-led customer conversations about availability and bookings
- businesses with 1-50 staff who dispatch jobs through ServiceM8

Not a fit:

- CRM-heavy workflows (use HubSpot, Pipedrive integrations instead)
- e-commerce (use Shopify integration)
- restaurant reservations (use SevenRooms integration)
- multi-location routing inside one ServiceM8 account

2. Before You Start

Make sure you have:

- access to the Newo project builder for your project
- permission to edit project settings and publish
- a ServiceM8 account (free trial works for testing)
- an API Key from ServiceM8

Who Usually Provides What

Role	Usually provides
Business owner / manager	ServiceM8 account access, staff list, service categories, business hours
ServiceM8 admin	API Key (Settings → API Keys), default staff UUID
Newo implementer / PM	Newo setup, feature toggles, publish, test plan, go-live validation

You will need these values before setup:

- **API Key** (from ServiceM8 Settings → API Keys)
- **Default Staff UUID** (from ServiceM8 staff list or API)
- **Default Category UUID** (optional, from ServiceM8 categories)

2.1 Setting Up a ServiceM8 Account from Scratch

If the business doesn't have a ServiceM8 account yet:

1. **Sign up** at servicem8.com/sign-up-free — free trial, no credit card
2. **Add staff members**: in ServiceM8, add at least 1-2 technicians/plumbers with name + email

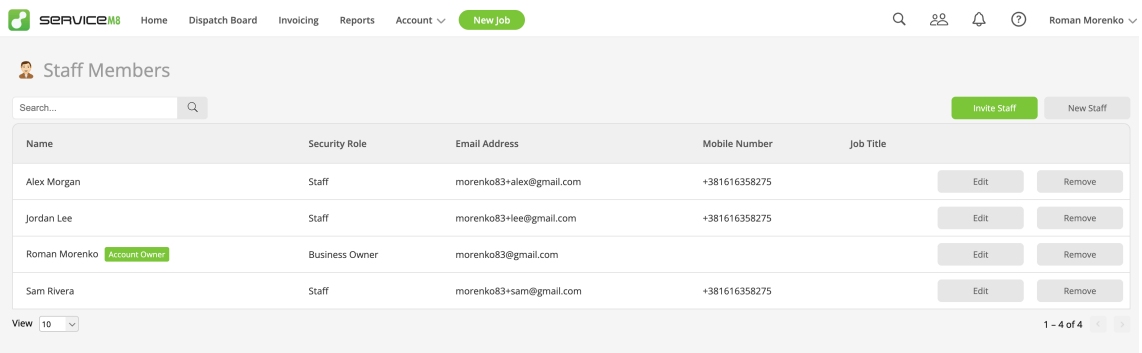

Figure S07. Staff list in ServiceM8.

3. **Add job categories**: go to **Materials & Services** → add categories matching the business services (e.g., "Plumbing Repair", "Water Heater", "Drain Cleaning")
4. **Generate API Key**: go to **Settings → API Keys** → Generate new key

> ***Note about allocation windows:*** *ServiceM8 creates default allocation windows (Morning, Afternoon, Business Hours) automatically when you sign up. No action needed — the integration uses them as-is.*

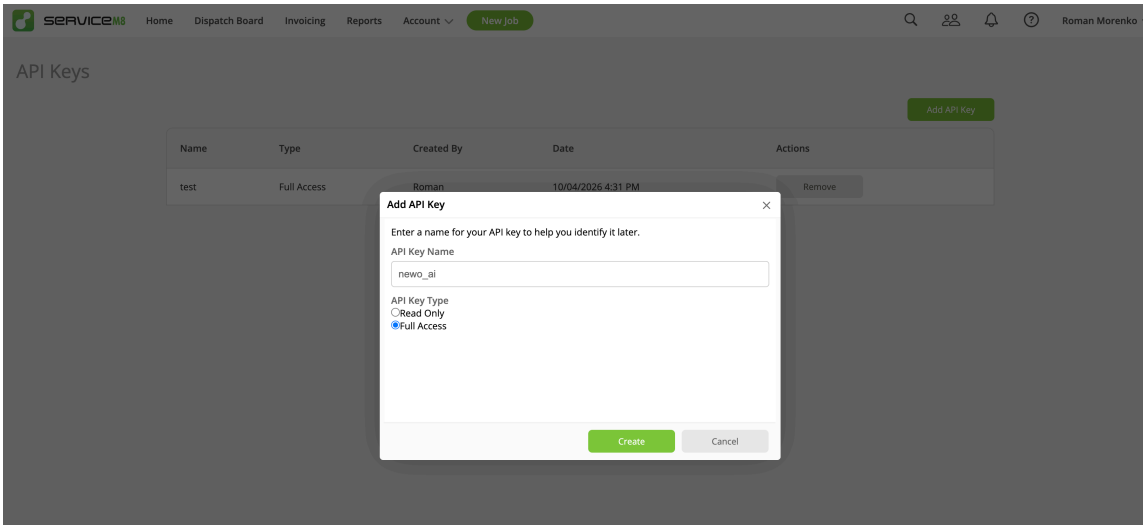

Figure S06. API Keys page in ServiceM8 Settings.

2.2 Getting the Default Staff UUID

The integration needs a default staff UUID to check availability and assign jobs. ServiceM8 UI does not show UUIDs, so you get them from Newo after first publish:

1. Set up just the **API Key** in Newo and publish
2. Open a test conversation (this triggers the first data refresh)
3. Go to **Builder → Attributes** → find `servicem8_staff`
4. It will contain a JSON list with all staff members and their UUIDs
5. Copy the `uuid` of the primary technician
6. Paste it into the **Default Staff UUID** field
7. Publish again

> ***Note:*** *ServiceM8 UI does not expose UUIDs anywhere. The auto-populated* `servicem8_staff` *attribute after publish is the easiest way to get them.*

2.3 Getting the Default Category UUID (optional)

Same approach — after publish, find `servicem8_categories` attribute in Newo. Copy the `uuid` of the main service category. If not set, jobs will be created without a category.

3. Connecting for the First Time

3.1 Configure ServiceM8 in Newo

1. Open your project in Newo.
2. Go to **Builder → Attributes**.
3. Find the **ServiceM8 Settings** section.
4. Fill in:
    - **API Key** — from ServiceM8 Settings → API Keys
    - **Default Staff UUID** — UUID of the primary staff member

- **Default Category UUID** — (optional) UUID of the default job category

5. Review the feature toggles:
    - **Enable Availability Check** — On
    - **Enable Booking** — On
    - **Enable Cancellation** — On
    - **Enable Reschedule** — On
    - **Enable Job Lookup** — On
    - **Enable Quote Creation** — On
    - **Setup Canvas Scenarios** — On
6. Click **Save**.
7. Click **Publish All**.

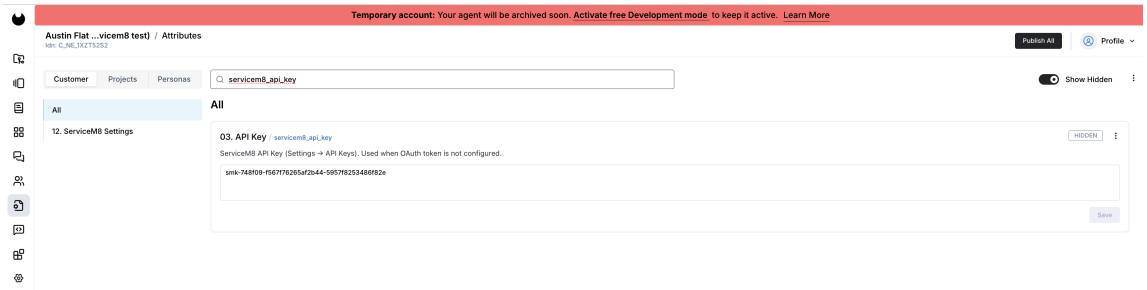

Figure S01. The API Key field in ServiceM8 Settings.

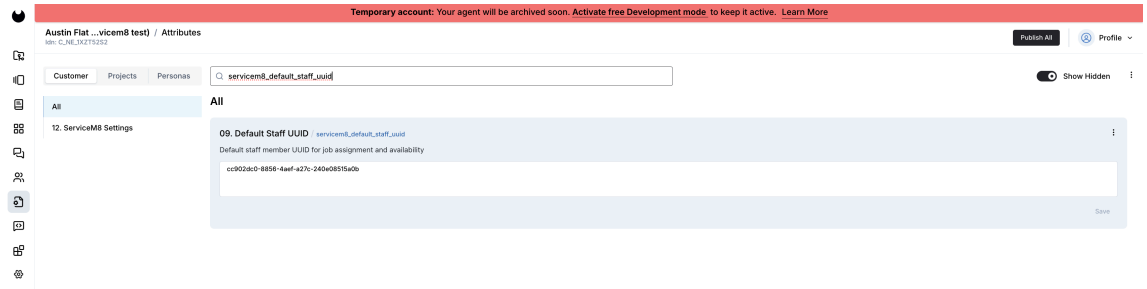

Figure S02. The Default Staff UUID field.

Figure S03. Feature toggles for ServiceM8 capabilities.

3.2 What Happens After Publish

When you publish:

- Newo creates the `servicem8_connector` HTTP connector
- creates the setup persona
- registers 6 custom AI tools (availability, booking, cancel, reschedule, lookup, quote)
- optionally auto-adds FSM scenarios to Canvas (booking, cancellation, job lookup)

On the first conversation (`session_started`):

- refreshes staff directory, job categories, and materials catalog from ServiceM8

3.3 What "Successful Setup" Looks Like

After setup is complete, you should expect:

- `servicem8_staff` has a non-empty JSON value (staff list)
- `servicem8_categories` has a non-empty JSON value
- the project can run an availability check without errors
- the ServiceM8 settings stay saved after refresh
- if Canvas auto-setup is enabled, FSM scenarios appear in Canvas

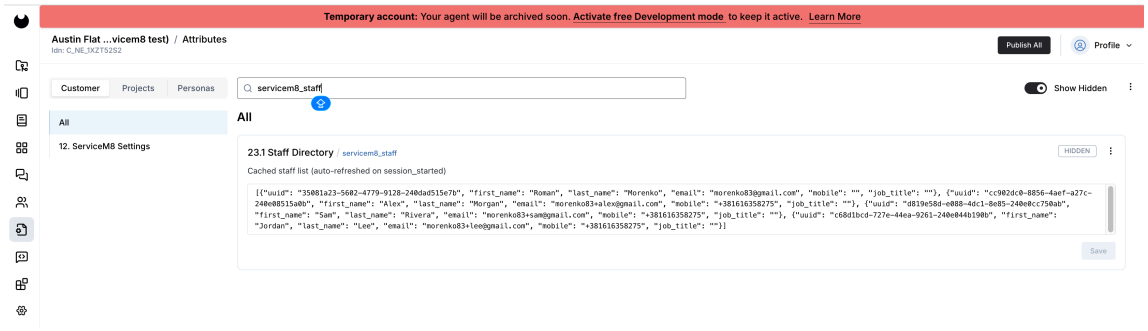

Figure S05. After successful publish — `servicem8_staff` populated with staff JSON.

3.4 Quick Smoke Test

1. Open a test conversation with the agent
2. Say: *"I need a plumber tomorrow. What times are available?"*
3. The agent should respond with available time slots (e.g., 8 AM to 5 PM)
4. If you get slots → setup is working

4. Settings Reference

All settings are in **Builder → Attributes → ServiceM8 Settings**.

4.1 Connection Settings

Setting	What it does	Required
Base URL	Root URL for the ServiceM8 API (`https://api.servicem8.com/api_1.0`)	Yes (auto-filled)
API Key	Sent in `X-API-Key` header for every API call	Yes
Default Staff UUID	Staff member used for availability checks and job assignment	Yes
Default Category UUID	Job category assigned to new jobs	Optional
Activity Duration (minutes)	Default job duration when scheduling (default: 60)	Optional

4.2 Feature Toggles

Setting	Default	What it controls
Enable Availability Check	On	Allows the agent to check staff availability
Enable Booking	On	Allows the agent to create field service jobs
Enable Cancellation	On	Allows the agent to cancel jobs
Enable Reschedule	On	Allows the agent to reschedule jobs
Enable Job Lookup	On	Allows the agent to look up job status
Enable Quote Creation	On	Allows the agent to create price quotes
Setup Canvas Scenarios	On	Adds FSM conversation scenarios to Canvas on publish

4.3 Technical / Auto-filled Settings

Setting	Default	What it does
Static Data Refresh Interval	`3600`	Seconds between staff/category/material refreshes (default: 1 hour)
Static Data Last Updated	Auto	Timestamp of the most recent background refresh

Staff Directory	Auto	Cached JSON list of active staff members
Job Categories	Auto	Cached JSON list of active job categories
Materials Catalog	Auto	Cached JSON list of active materials
Setup Persona ID	Auto	Persona used during setup operations

5. What the AI Agent Can Do

5.1 Check Availability

The AI checks ServiceM8 availability for a requested date:

- fetches allocation windows (shift templates)
- fetches already-scheduled activities for the staff
- calculates free slots by subtracting busy periods from business hours

The integration calls:

- `GET /allocationwindow.json` — shift templates
- `GET /jobactivity.json` — scheduled activities

What the agent receives:

- list of available time slots for the requested date (e.g., 8 AM, 9 AM, 10 AM...)

5.2 Create a Job (Booking)

The AI creates a field service job after collecting:

- service description (what the customer needs)
- preferred date and time
- customer name, phone, email
- service address

The integration calls:

- `GET /search.json` — find existing client
- `POST /company.json` — create new client (if not found)
- `GET /company.json` — resolve client UUID
- `POST /job.json` — create the job
- `GET /job.json` — resolve job UUID
- `POST /jobactivity.json` — schedule the activity

On success, the agent:

- stores booking metadata in the persona `bookings` attribute
- confirms the booking to the customer with date, time, and job reference

5.3 Look Up Job Status

The AI can look up an existing job when the customer asks about it.

The integration:

1. reads the customer's bookings from persona attributes

2. matches the customer's description against stored bookings
3. calls `GET /job/{uuid}.json` to get current status
4. calls `GET /jobactivity.json` to get schedule details
5. returns status, scheduled date/time, and description to the customer

No UUID needed from the customer — the agent matches by date/time/description from persona bookings.

5.4 Cancel a Job

The AI cancels a job when the customer requests it.

The integration:

1. reads bookings from persona, matches by context
2. calls `POST /job/{uuid}.json` with status `Unsuccessful`
3. calls `GET /jobactivity.json` to find scheduled activities
4. calls `DELETE /jobactivity/{uuid}.json` to remove them

No UUID needed from the customer.

5.5 Reschedule a Job

The AI can move a job to a new date/time.

The integration:

1. reads bookings from persona, matches by context
2. finds the existing activity via `GET /jobactivity.json`
3. updates it via `POST /jobactivity/{uuid}.json` with new start/end dates

5.6 Create a Quote

The AI can create a price quote / estimate for a customer.

The integration reuses the client search/create chain, then:

- calls `POST /job.json` with status `Quote`

5.7 Auto-Setup Canvas Scenarios

If Canvas auto-setup is enabled, ServiceM8 inserts FSM-specific scenarios:

- **Book a Field Service Job** scenario + intent
- **Cancel a Field Service Job** scenario + intent
- **Look Up Job Status** scenario + intent

These replace the default NAF appointment scenarios with field-service-specific language and code-phrases.

5.8 Refresh Static Data

On each new conversation, ServiceM8 refreshes (if stale):

- **Staff directory** — names, UUIDs, contact info
- **Job categories** — names, UUIDs
- **Materials catalog** — names, prices, UUIDs

This happens automatically via `StaticDataFlow` on `session_started`.

6. Supported Features

Feature	Supported	Notes
Check availability	Yes	Uses allocation windows + activity schedule
Create job (booking)	Yes	Full chain: search/create client → create job → schedule activity
Cancel job	Yes	Sets status to Unsuccessful, removes activities
Reschedule job	Yes	Updates activity schedule
Look up job status	Yes	Returns status, date, time, description
Create quote	Yes	Creates Quote-status job
Canvas auto-setup	Yes	Inserts FSM-specific scenarios
Static data refresh	Yes	Staff, categories, materials
Webhook notifications	No	Not needed — removed by design
Email/SMS from ServiceM8	No	Requires OAuth, removed by design
Multi-location	No	One ServiceM8 account per project
Custom fields	No	Not implemented

7. Technical Architecture

7.1 Custom Newo Tools

Tool	Trigger code-phrase
`servicem8_check_availability`	"Let me check what times are available..."
`servicem8_create_job`	"I'm booking that service for you now..."
`servicem8_cancel_job`	"I'll cancel that job now..."
`servicem8_reschedule_job`	"Let me reschedule that for you..."
`servicem8_lookup_job`	"Let me look up your job..."
`servicem8_create_quote`	"Let me put together a quote..."

7.2 Internal Events

Event	Purpose
`servicem8_execute_request`	Send an authenticated HTTP request through NetworkFlow
`servicem8_result_success`	Route successful API responses back to ApiFlow

`servicem8_result_error`	Route failed API responses back to ApiFlow
`servicem8_availability_success`	Availability slots ready for the agent
`servicem8_booking_success`	Job booked successfully
`servicem8_cancellation_success`	Job cancelled successfully
`servicem8_lookup_success`	Job details retrieved

7.3 High-Level Sequence

```
sequenceDiagram
    participant Customer
    participant ConvoAgent
    participant ServiceM8Integration
    participant ServiceM8API

    Customer->>ConvoAgent: Ask about availability / booking / cancel
    ConvoAgent->>ServiceM8Integration: Trigger custom tool event
    ServiceM8Integration->>ServiceM8Integration: Extract params from conversation
(Gen LLM)
    ServiceM8Integration->>ServiceM8API: Authenticated request (X-API-Key)
    ServiceM8API-->>ServiceM8Integration: JSON response
    ServiceM8Integration->>ServiceM8Integration: Parse, resolve UUIDs, update
persona
    ServiceM8Integration-->>ConvoAgent: Result via urgent_message
    ConvoAgent-->>Customer: Slots, confirmation, cancellation, or status
```

7.4 Three-Layer Architecture

```
Entry Layer          API Layer (ApiFlow)         Network Layer
_____________       ___________________          ______________

AvailabilityFlow ──► CheckAvailability chain ──► _executeRequestSkill
BookingFlow ────────► Search/Create/Job chain ──► SendCommand(send_request)
CancellationFlow ──► Cancel chain ──────────────► servicem8_connector
RescheduleFlow ─────► Reschedule chain                    │
ExistingClientFlow ► Lookup chain              HTTP GET/POST/DELETE
QuoteFlow ──────────► Quote chain                         │
                                               ServiceM8 REST API
```

8. How to Test That Everything Works

Recommended Test Order

1. **Availability**: proves API key auth and staff data work
2. **Booking**: proves full chain (search → create client → job → activity)
3. **Cancellation**: proves lookup from persona bookings + cancel chain
4. **Reschedule**: proves activity update
5. **Quote**: proves quote-status job creation

8.1 After Setup

After clicking **Publish All**, confirm:

- [ ] no publish errors appear
- [ ] `servicem8_staff` has a non-empty JSON value (wait for first `session_started`)
- [ ] `servicem8_categories` has a non-empty JSON value
- [ ] feature toggles have the intended values

8.2 Test Availability

1. Start a test conversation
2. Say: *"I need a plumber tomorrow at 123 Main St. What times are available?"*
3. Agent should return time slots (e.g., 8 AM to 5 PM with gaps)
4. If slots appear → availability is working

8.3 Test Booking

1. Pick a slot from the availability results
2. Provide name, phone, and email when asked
3. Agent should confirm: *"Your job has been booked for [date] at [time]"*
4. Verify the job appears in ServiceM8 (Jobs list)

8.4 Test Cancellation

1. In the same conversation, say: *"Actually, I need to cancel that appointment"*
2. Agent should find the booking automatically (no UUID needed)
3. Agent asks: *"Are you sure?"*
4. Confirm → Agent says: *"Your job has been cancelled"*
5. Verify job status is `Unsuccessful` in ServiceM8

8.5 Test Reschedule

1. Book a job first
2. Say: *"Can you move my appointment to next Friday at 2 PM?"*
3. Agent should update the schedule
4. Verify the activity dates changed in ServiceM8

8.6 Test Quote

1. Ask: *"Can I get a quote for installing a new water heater?"*
2. Provide details when asked
3. Agent should create a Quote-status job in ServiceM8

9. FAQ and Troubleshooting

Why does the agent say "no slots available"?

Check:

- ServiceM8 has **allocation windows** configured (Settings → Allocation Windows)
- There is at least one active staff member
- `servicem8_default_staff_uuid` is set correctly

Why does booking fail with "scheduling system error"?

Most commonly:

- **API Key is empty** — check `servicem8_api_key` attribute
- **API Key is invalid** — generate a new one in ServiceM8 Settings → API Keys
- **ServiceM8 account is on a restricted plan** — try generating a new key

Why can't the agent find my booking for cancellation?

The agent reads bookings from the **persona attribute**, not from ServiceM8 directly. If the booking was created outside the AI conversation (e.g., in ServiceM8 UI), the agent won't know about it. Bookings must be created through the AI to appear in the cancel/reschedule flow.

Do I need OAuth?

No. This integration uses API Key authentication only. No OAuth setup is required.

Does this work with ServiceM8 free trial?

Yes. All features work with the free trial account.

Can I use this for multiple locations?

Not in the current implementation. One ServiceM8 account per Newo project.

10. Manager Rollout Summary

This integration is a good fit if you want the AI to handle:

- scheduling availability questions from customers
- field service job creation (booking)
- job changes (reschedule) and cancellations
- job status inquiries
- price quote creation

This integration is **not** currently positioned as:

- a CRM replacement
- an invoicing system
- a customer notification system (no email/SMS from ServiceM8)

Recommended rollout order:

1. Set up ServiceM8 account (or use existing)
2. Generate API Key
3. Configure in Newo (API Key + Default Staff UUID)
4. Publish
5. Test availability → booking → cancel
6. Review Canvas scenarios
7. Go live

Go-Live Recommendation

For the cleanest launch:

- start with one service type (e.g., plumbing only)
- keep all core features enabled unless there is a clear business reason not to
- run at least one real booking and cancellation before launch day
- have a fallback human dispatcher ready for the first live shift